AI для .NET-розробника

Том 3 · Налаштування й .md-файли

Розділи 7–9

Розділ 7. Файли інструкцій

7.1. Ідея

Замість повторювати в кожному чаті «використовуй .NET 9, xUnit, не чіпай міграції», ви один раз записуєте правила у Markdown-файл у репозиторії. Інструмент автоматично додає його до контексту. Це «постійний системний промпт проєкту».

7.2. Карта файлів за інструментами

Файл	Де лежить	Хто читає
`copilot-instructions.md`	`.github/`	GitHub Copilot (VS, VS Code, github.com, CLI, cloud agent)
`*.instructions.md`	`.github/instructions/`	Copilot — правила для певних файлів за маскою `applyTo`
`AGENTS.md`	Корінь репозиторію (і підпапки)	Відкритий формат: Copilot, Codex, Cursor та інші агенти
`CLAUDE.md`	Корінь (і підпапки, домашня `~/.claude/`)	Claude Code
`GEMINI.md`	Корінь	Gemini CLI
`.cursor/rules/*`	Корінь	Cursor

> Підтримка форматів і пріоритет, коли існує кілька файлів одночасно, залежать від продукту й версії. Перевіряйте документацію конкретного інструмента. Практична порада: тримайте один основний файл (наприклад, `AGENTS.md`), а в інших — посилання на нього.

7.3. Що писати

Пишіть	Не пишіть
Команди збірки, тестів, лінту	Загальні істини («пиши чистий код»)
Версію .NET, стек, структуру рішення	Довгі пояснення мовою C#
Конвенції іменування й архітектурні правила	Те, що вже видно з коду й EditorConfig
Заборони (секрети, ручне редагування міграцій)	Секрети та реальні паролі
Як писати тести й що вважати «готово»	Файл на 1000 рядків — модель «втрачає» важливе

Орієнтир: 20–100 рядків, чіткі короткі правила.

7.4. Приклад `.github/copilot-instructions.md` для .NET

```
# Проєкт TodoApi

## Стек
- .NET 9, ASP.NET Core Minimal API, EF Core (SQLite), xUnit + Moq.
- Рішення: src/TodoApi (API), tests/TodoApi.Tests.

## Команди
- Збірка: `dotnet build`
- Тести: `dotnet test`
- Формат: `dotnet format`

## Правила коду
- Nullable reference types увімкнені; не вимикай попередження.
- Усі I/O-методи async і приймають CancellationToken.
- Використовуй ILogger<T>, не Console.WriteLine.
- DTO — record-типи; сутності EF не повертай з API напряму.
- Іменування: PascalCase для публічного, _camelCase для приватних полів.

## Заборонено
- Секрети в коді: тільки User Secrets / змінні середовища.
- Ручне редагування файлів у Migrations/.
- Додавати NuGet-пакети без явного прохання.

## Тести
- Кожна нова публічна поведінка має тест; назва: Метод_Сценарій_Результат.
```

7.5. Path-specific інструкції

Файл `.github/instructions/tests.instructions.md` із YAML-заголовком застосовується лише до файлів за маскою:

```
---
applyTo: "tests/**/*.cs"
---
- Використовуй xUnit і FluentAssertions.
- Один Assert-концепт на тест; без логіки (if/for) у тестах.
- Мокай лише зовнішні залежності.
```

7.6. AGENTS.md

Формат без жорсткої схеми — просто Markdown з розділами: огляд, команди, стиль, тестування, безпека. Для моноріпозиторіїв можна класти додаткові `AGENTS.md` у підпапки — найближчий до змінюваного файлу має пріоритет (у більшості агентів).

7.7. Як перевірити, що інструкції працюють

- У відповіді чату є список «References/Used references» — там має бути ваш файл.
- Попросіть: «Які інструкції проєкту ти бачиш?».
- Напишіть тестовий запит, що порушує правило, і подивіться, чи агент опирається.

Інструкції — **рекомендація**, а не гарантія. Для жорсткого контролю використовуйте hooks, аналізатори, CI.

> **Завдання 7.** Створіть `.github/copilot-instructions.md` для TodoApi (за прикладом 7.4) та `tests.instructions.md`. Попросіть агента додати метод із тестом і перевірте, чи дотримано правила.

Розділ 8. Prompt files і Skills

8.1. Prompt files — збережені запити

Це Markdown-файли, які ви викликаєте командою `/ім'я` у чаті. Вони замінюють копіювання одного й того ж промпту.

Розташування: `.github/prompts/назва.prompt.md`.

```
---
description: Створити xUnit-тести для вибраного класу
agent: agent
---
Напиши xUnit-тести для ${selection} (або для #file, якщо нічого не виділено).
Покрий: щасливий шлях, null/порожні значення, винятки.
Після створення виконай `dotnet test` і виправ помилки.
```

Виклик у чаті: `/tests` (назва файлу без суфікса). Змінні (`${selection}`, `${file}`) та поля заголовка (`agent` / `mode`, `model`, `tools`) залежать від версії інструмента — звіряйтеся з документацією.

8.2. Skills — навички агента

Skill — це папка з обов'язковим файлом `SKILL.md` і, за потреби, скриптами, шаблонами, прикладами. Агент бачить лише **ім'я й опис** кожного скіла; повний текст підвантажується, коли задача підходить. Це економить контекст («progressive disclosure»).

```
.github/skills/
  ef-migration/
    SKILL.md
    templates/migration-checklist.md
    scripts/check-migrations.ps1
```

```
---
name: ef-migration
description: Створення й перевірка міграцій EF Core. Використовуй, коли користувач
просить змінити схему БД або додати таблицю/колонку.
---
# Міграції EF Core
1. Змінити сутність у src/TodoApi/Data.
2. `dotnet ef migrations add <Name> -p src/TodoApi`
3. Переглянути згенерований файл: немає DROP без підтвердження.
4. `dotnet ef database update` лише на локальній БД.
5. Додати запис у CHANGELOG.
```

Поширені місця зберігання: проєктні `.github/skills/` або `.claude/skills/`, особисті — у домашній директорії (`~/.copilot/skills`, `~/.claude/skills`). Формат `SKILL.md` — відкритий стандарт, тож ті самі скіли часто працюють у різних агентах.

8.3. Як написати хороший опис скіла

- Опис — це «тригер»: *що робить* і *коли використовувати*.
- Включайте ключові слова, які вживатиме користувач («міграція», «схема БД»).
- Тіло — покрокова процедура, а не есе.
- Довгі довідники виносьте в окремі файли й посилайтеся на них.

8.4. Порівняння механізмів

Механізм	Коли підвантажується	Для чого
copilot-instructions / AGENTS.md	Завжди	Загальні правила проєкту
`*.instructions.md`	Коли працюєте з файлами за маскою	Правила для частини коду
Prompt file	Коли ви викликаєте `/команду`	Повторювані запити
Skill	Коли агент вирішив, що це доречно (за описом)	Спеціалізовані процедури зі скриптами
Custom agent	Коли ви обрали персону	Роль + набір інструментів
MCP-сервер	Коли агент викликає його інструмент	Доступ до зовнішніх систем

> Правило вибору: «завжди» → інструкції; «я запускаю вручну» → prompt file; «агент має сам здогадатися» → skill; «потрібні дії у зовнішній системі» → MCP.

> **Завдання 8.** Створіть prompt file `/review` (рев'ю поточних змін за чекліст: null-safety, async, безпека, тести) та skill `ef-migration`. Викличте обидва.

Розділ 9. Custom agents, hooks і дозволи

9.1. Custom agents (раніше «chat modes»)

Це збережена **персона**: інструкції + дозволений набір інструментів (+ за потреби модель). Файл: `.github/agents/назва.agent.md`.

```
---
name: dotnet-reviewer
description: Суворий рев'юер .NET-коду, змін не вносить
tools: ['search', 'read']
---
Ти — досвідчений рев'юер .NET. Перевіряй: null-safety, async/await,
CancellationToken, dispose ресурсів, SQL-ін'єкції, секрети, покриття тестами.
Файли НЕ змінюй. Видавай список зауважень за пріоритетом із посиланнями на рядки.
```

Приклади корисних агентів: **Planner** (лише читає й планує), **Reviewer**, **Test-writer**, **Security auditor**, **Docs writer**. Обмеження інструментів — головна перевага: рев'юер фізично не зможе щось зламати.

9.2. Субагенти

Основний агент може делегувати підзадачу іншому агенту в **окремому контексті**: той виконує дослідження (читає 50 файлів) і повертає лише короткий підсумок. Основна розмова не засмічується. Ви часто можете вказати, який агент підходить для якої підзадачі.

9.3. Hooks

Hooks — команди/скрипти, що запускаються на подіях життєвого циклу агента: початок сесії, *перед* викликом інструмента (можна заблокувати), *після* (наприклад, форматувати файл). Конфігурація — JSON-файл у репозиторії (наприклад, у `.github/hooks/`); точний формат і набір подій залежать від продукту, тому дивіться актуальну документацію.

Ідея hook-а	Користь
Після редагування `.cs` → `dotnet format`	Єдиний стиль без уваги моделі
Перед запуском команди з `rm -rf`, `git push --force` → блок	Захист від руйнівних дій
Перед записом у `Migrations/` → відмова	Жорстке виконання заборон
Логування всіх викликів інструментів	Аудит

Різниця з інструкціями: інструкції — прохання, hooks — **гарантоване виконання** детермінованого коду.

9.4. Дозволи й рівні автономності

Режим	Поведінка	Ризик
Підтвердження кожної дії	Агент питає перед командою/записом	Низький, але повільно
Allow-list команд	Дозволені команди (`dotnet build`, `dotnet test`) виконуються без питань	Помірний — рекомендований баланс
Auto-approve / autopilot	Усе без питань	Високий; лише в ізольованому середовищі

- Працюйте в гілці, коммітьте часто — `git` це ваша «кнопка скасування».
- Для ризикових експериментів використовуйте контейнер/ВМ/Codespace.
- **Prompt injection:** агент, що читає чужі issue, веб-сторінки чи залежності, може отримати шкідливі інструкції. Не давайте йому одночасно доступ до секретів, інтернету й запису без нагляду.

9.5. Рекомендована структура репозиторію

```
.github/
  copilot-instructions.md
  instructions/ tests.instructions.md, api.instructions.md
  prompts/      review.prompt.md, tests.prompt.md
  agents/       dotnet-reviewer.agent.md, planner.agent.md
  skills/       ef-migration/SKILL.md
  hooks/        format-after-edit.json
AGENTS.md
.editorconfig
```

> **Завдання 9.** Створіть агента `dotnet-reviewer` (лише читання) і спробуйте попросити його змінити файл — переконайтеся, що він не може. Сформулюйте, які 2 hooks були б корисні у вашому проєкті.

> **Далі — Том 4:** MCP, агентна робота, Copilot CLI та cloud agent.